

Models of the motor system

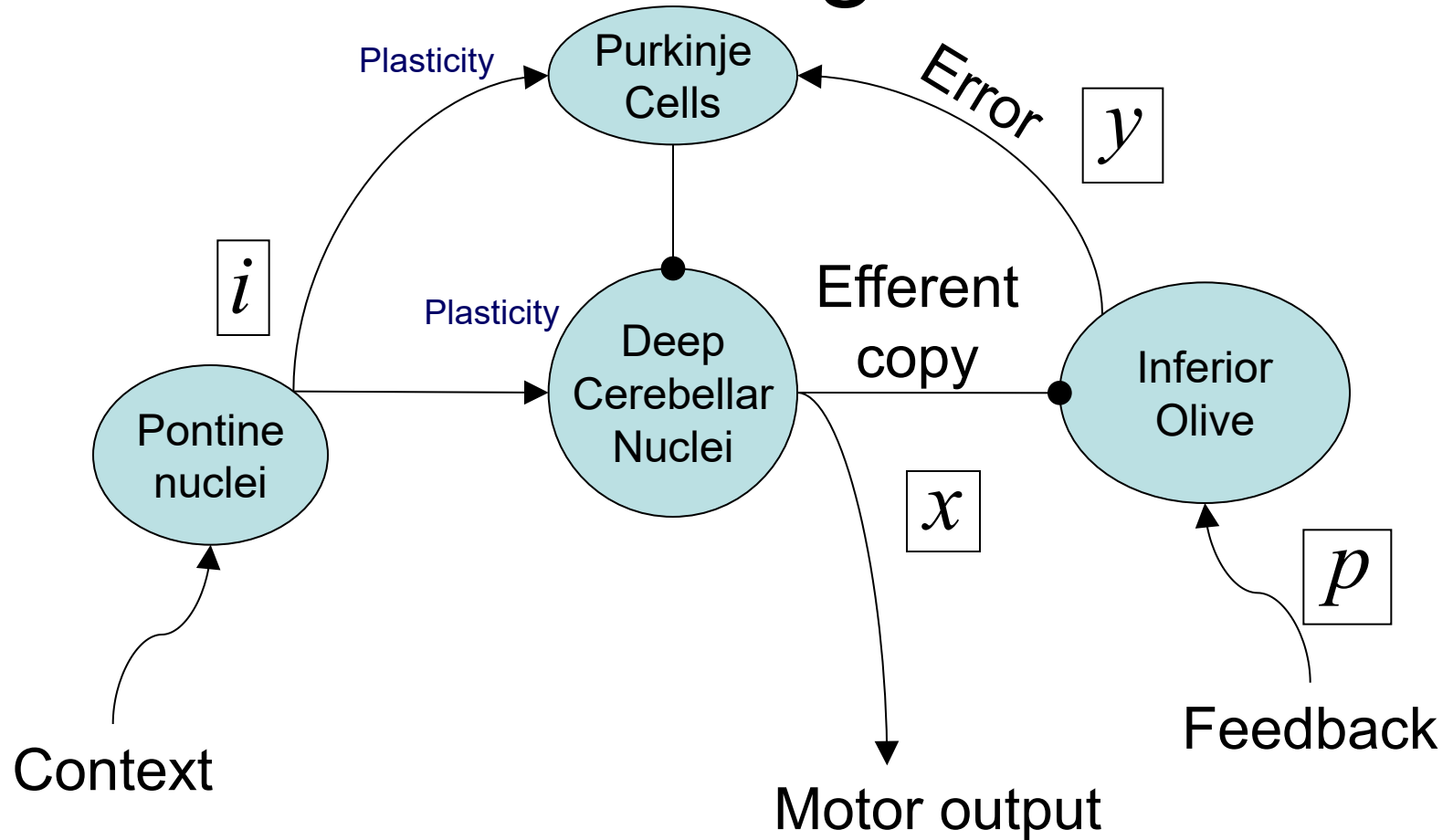
Fall 2021

Opher Donchin

Part 1

- Error based and reward based learning

A model of error based learning



Obligatory sports video



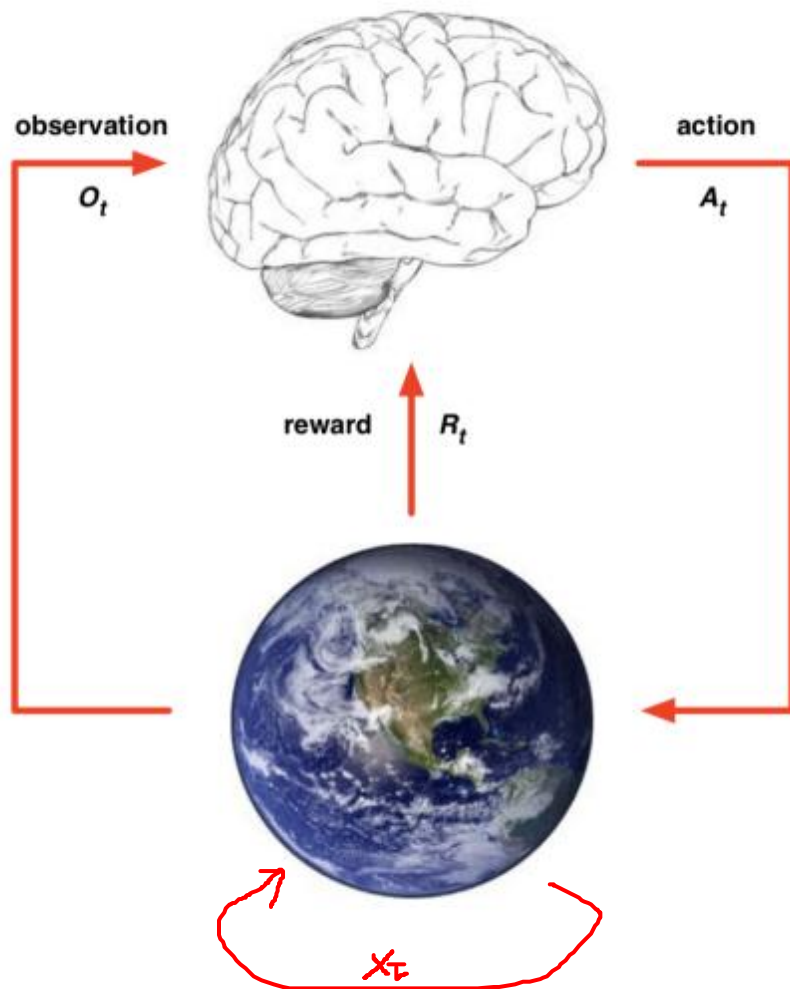
<https://www.youtube.com/embed/8dLx53NanQ0>

Error based learning may not work here



<https://www.youtube.com/embed/S9KE2R92pSg>

A model of reinforcement learning



- At each time step, t
 - The agent
 - Executes action A_t
 - The environment
 - Updates state x_t
 - Causing observation O_t
 - Producing reward R_t

Part 2

- What is reinforcement

Reward

- Something an animal will seek out
 - Positively motivating to the animal

Reward examples

- Food for an experimental animal
- Points in a video game
- Performing desired movement

Complex rewards

- Food for an experimental animal
 - In minimum time
- Points in a video game
 - But not dying
 - And collecting prizes
- Performing desired movement
 - With minimal effort
 - Optimizing a functional
 - Under constraints
 - Minimizing task-relevant variability

Reinforcement

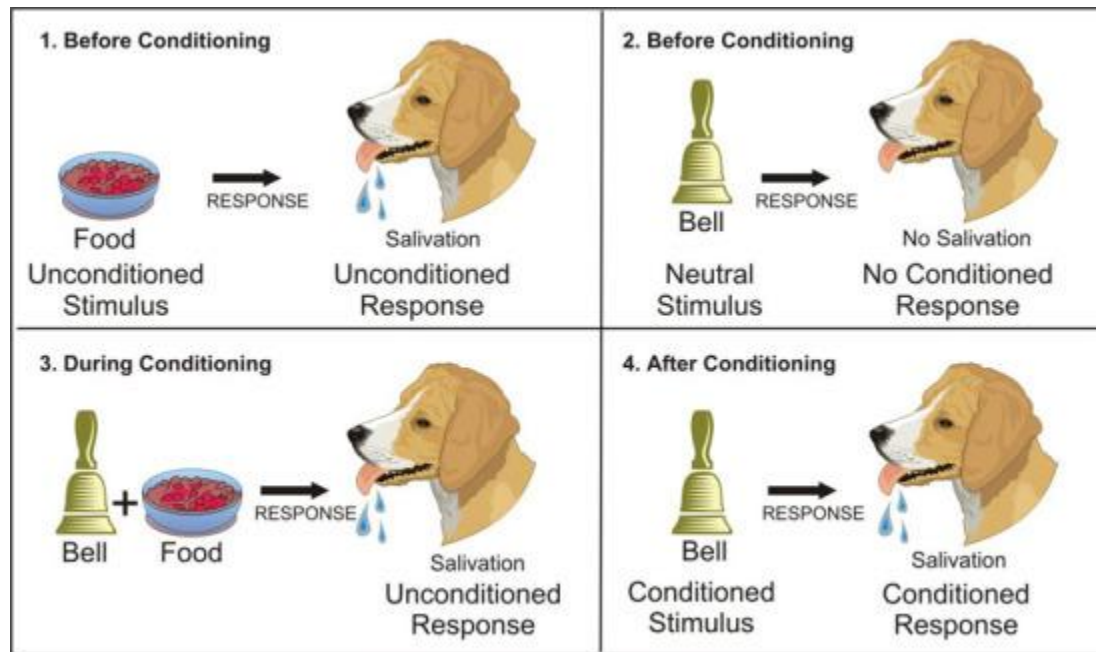
- **An environmental response**
 - To a behavior
 - That increases the chance of that behavior

Reward and reinforcement

- Reward
 - Something the animal seeks out
- Reinforcement
 - Something that promotes a behavior
- A reward, paired with a behavior
 - Becomes a reinforcement

Classical conditioning

- Pavlov's dog
 - “Learns” that the bell means food
 - Bell becomes rewarding



Classical Conditioning

Reinforcement and punishment

- Reinforcement
 - Increase behavior
 - Positive
 - Candy for the winner of a quiz
 - Negative
 - Free time if we get through the material
- Punishment
 - Decrease behavior
 - Positive
 - Do extra homework if late for class
 - Negative
 - Lose points for late submission

Part 3

- What is reinforcement learning

Reward based learning

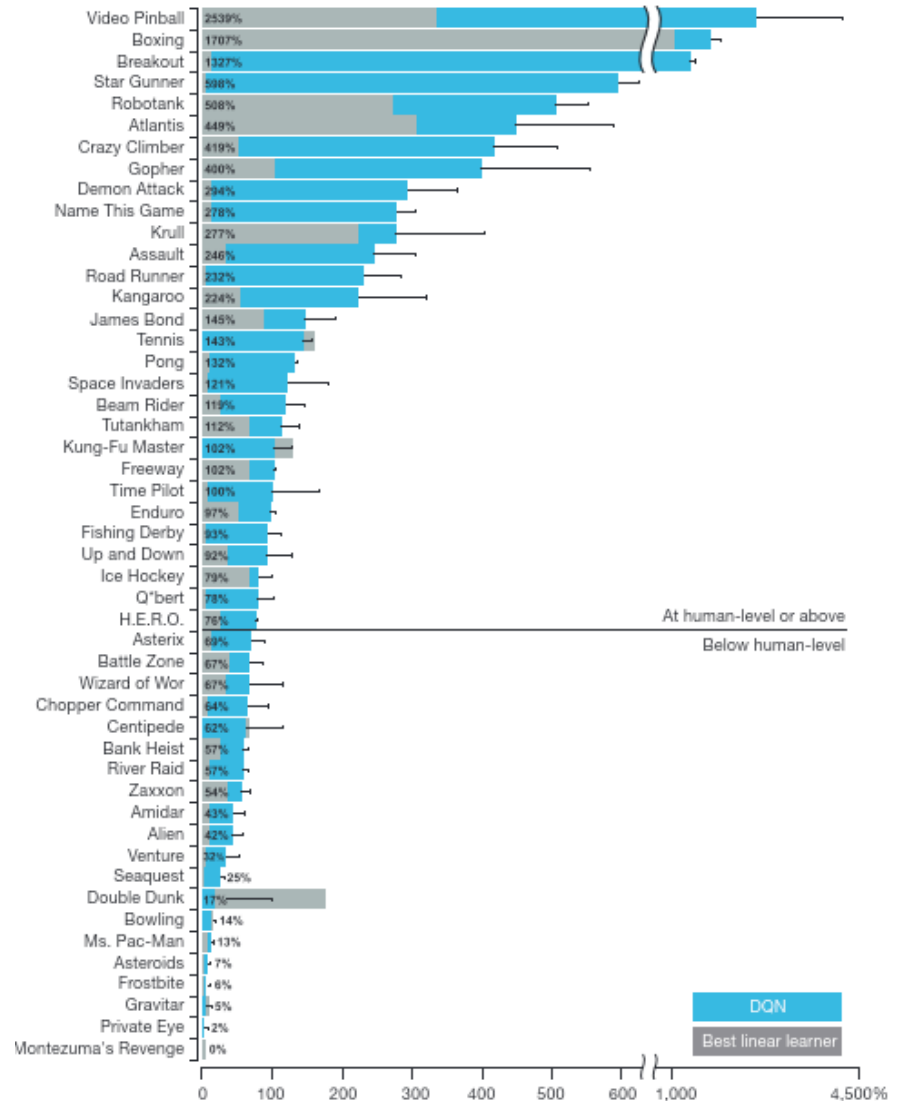
- Error based learning
 - Response to behavior is **signed** or **vectorial**
 - Indicates direction for change
 - Possibility for optimum
- Reward based learning
 - Response to behavior is **positive scalar**
 - Indicates relative success
 - No indication of direction for change
 - No indication of optimum

Reinforcement learning

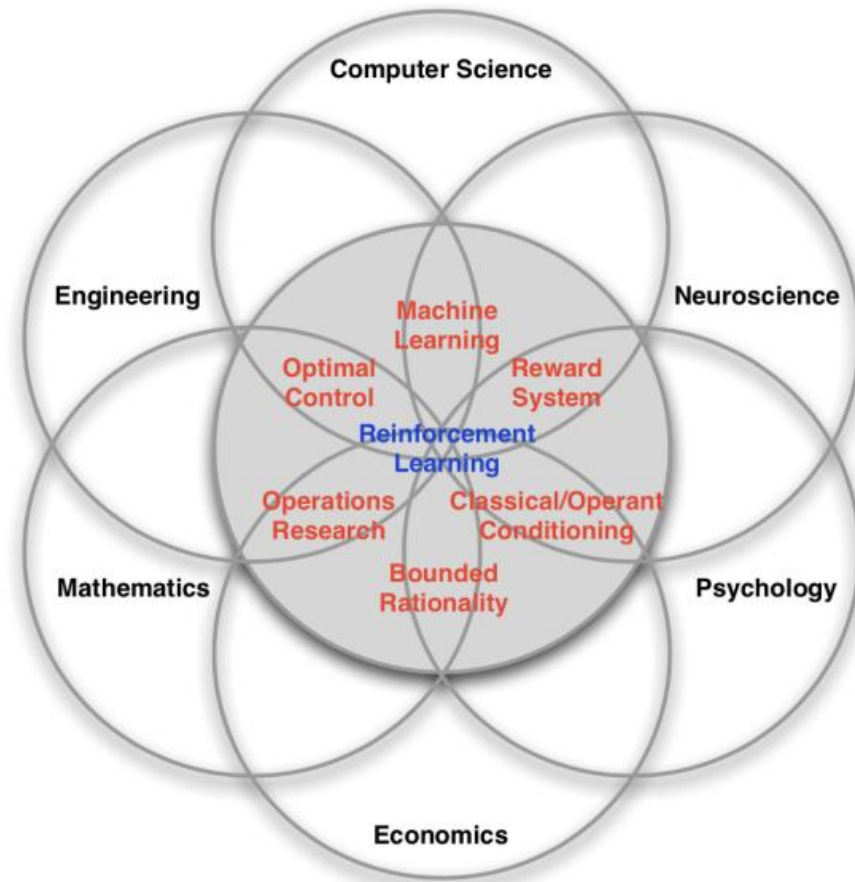
- Reward based
- Attempts to maximize reward over time
- Must predict rewards
- Must learn actions that maximize rewards

Heavily used in machine learning

- Reinforcement learning beats human players in video games
- Self-driving cars
- Robotics
- Medical diagnosis
- Basic to deep learning



Enormously influential



Learning to move robot arm

- Demo for python-based reinforcement learning toolbox



<https://www.youtube.com/watch?v=BZLX-Bilqzg>

Learning locomotion



<https://www.youtube.com/watch?v=BZLX-Bilqzg>

Part 4

- Algorithms of reinforcement learning

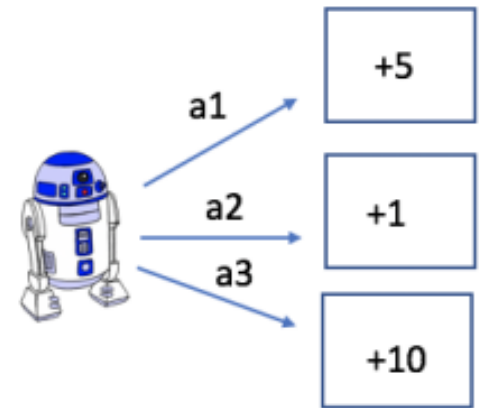
Maximize reward

- Learn rewards
- Choose action with best reward

$$a(t) = \arg \max_{a'} \hat{r}_t(a', x(t))$$

$$r(t) = g(a(t), x(t))$$

$$x(t+1) = f(a(t), x(t))$$



$$\hat{r}_{t+1}(a(t), x(t)) = \hat{r}_t(a(t), x(t)) + \delta \cdot (r(t) - \hat{r}_t(a(t), x(t)))$$

State transition function

- This could be given or learned
 - It doesn't have to be discrete

$$x(t+1) = f(a(t), x(t))$$

State transition		Action		
		a_1	a_2	a_3
State	x_1	$f(a_1, x_1)$		
	x_2			$f(a_3, x_2)$
	x_3			
	x_4			

True reward function

- This is unknown to the learner

$$r = g(a, x)$$

Reward		Action		
		a_1	a_2	a_3
State	x_1	$g(a_1, x_1)$		
	x_2			$g(a_3, x_2)$
	x_3			
	x_4			

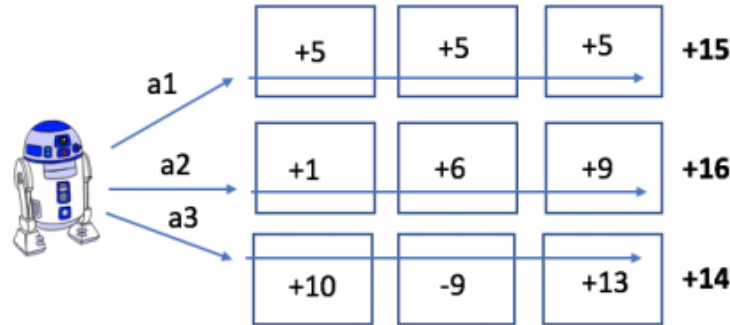
Estimated reward function

- Guessed by the learner
 - So far
 - And updated

$$\hat{r}_t(a(t), x(t)) = \hat{r}_t(a(t), x(t)) + \delta \cdot (r(t) - \hat{r}_t(a(t), x(t)))$$

Reward		Action		
		a_1	a_2	a_3
State	x_1	$\hat{r}_t(a_1, x_1)$		
	x_2			$\hat{r}_t(a_3, x_2)$
	x_3			
	x_4			

Maximize overall reward



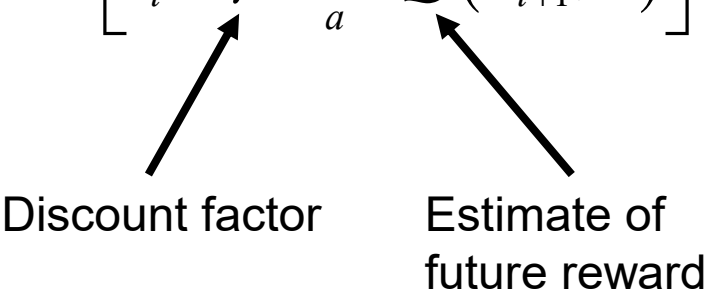
- Trade-off cumulative and immediate reward
 - Chess: position not pieces
- Reinforce predicted reward and not reward

Q-learning

Reward learning

$$\hat{r}_{t+1}(a(t), x(t)) = \hat{r}_t(a(t), x(t)) + \delta \cdot (r(t) - \hat{r}_t(a(t), x(t)))$$

Q-learning

$$Q_{t+1}(x_t, a_t) = (1 - \alpha) Q_t(x_t, a_t) + \alpha \left[r_t + \gamma \max_a Q(x_{t+1}, a) \right]$$


Discount factor

Estimate of future reward

- One of many popular reinforcement learning algorithms
- Update estimate of total reward
 - For actual reward received
 - This propagates backwards
- Make choices based on estimated total

Q-learning

$$Q_{t+1}(x_t, a_t) = (1 - \alpha) Q_t(x_t, a_t) + \alpha \left[r_t + \gamma \max_a Q(x_{t+1}, a) \right]$$

Discount factor

Estimate of
future reward

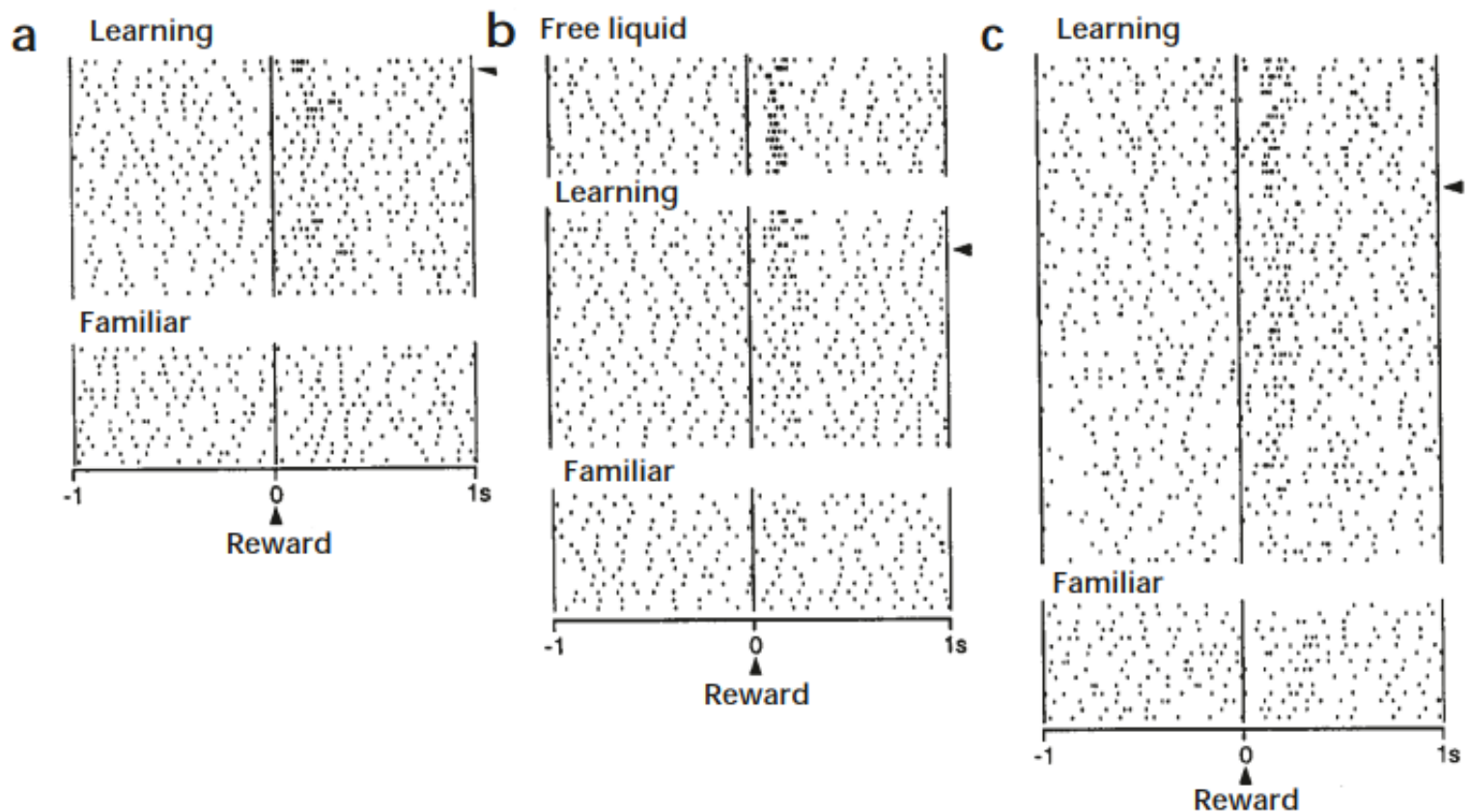
- Learning rate α affects how quickly things change
- Discount factor γ is tradeoff between immediate and future rewards

Part 5

- Biological reward prediction

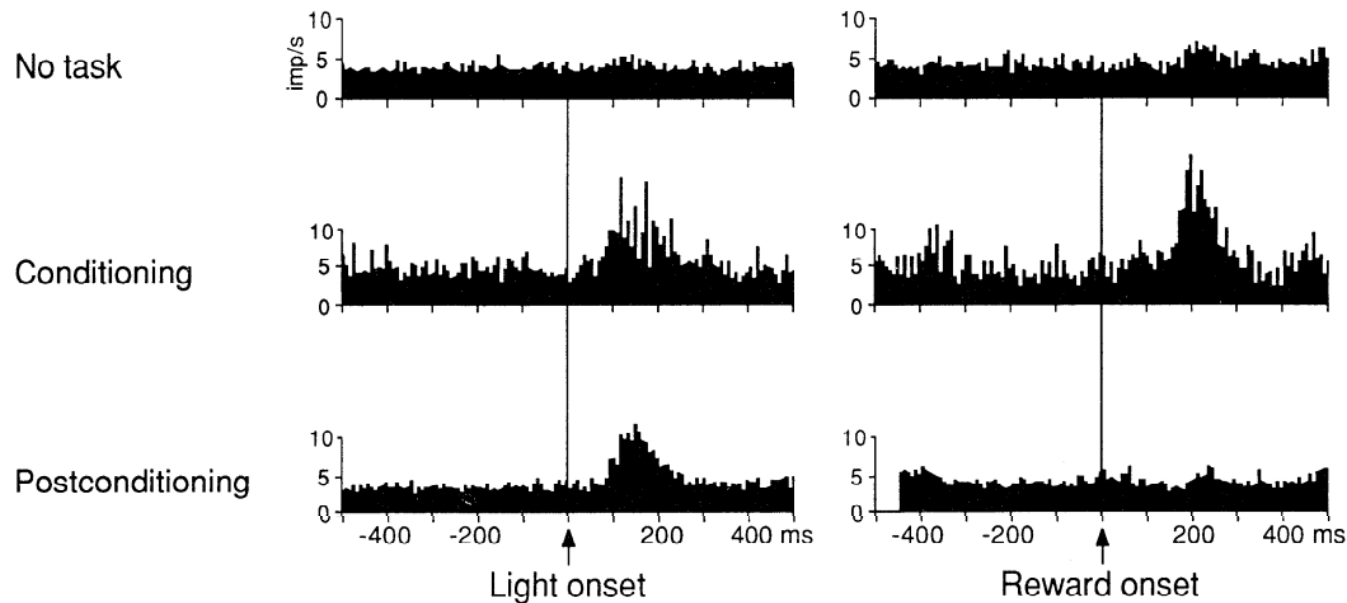
Dopamine neurons respond to reward

- But only when it is unexpected



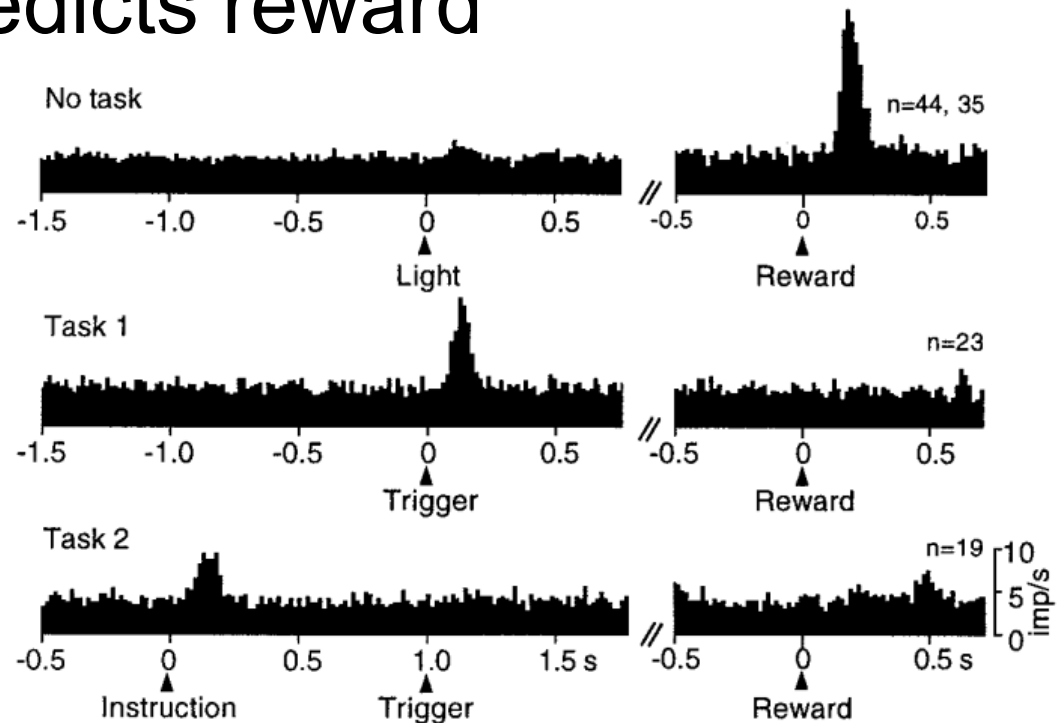
Dopamine response shifts to predictive stimulus

- Light predicts reward

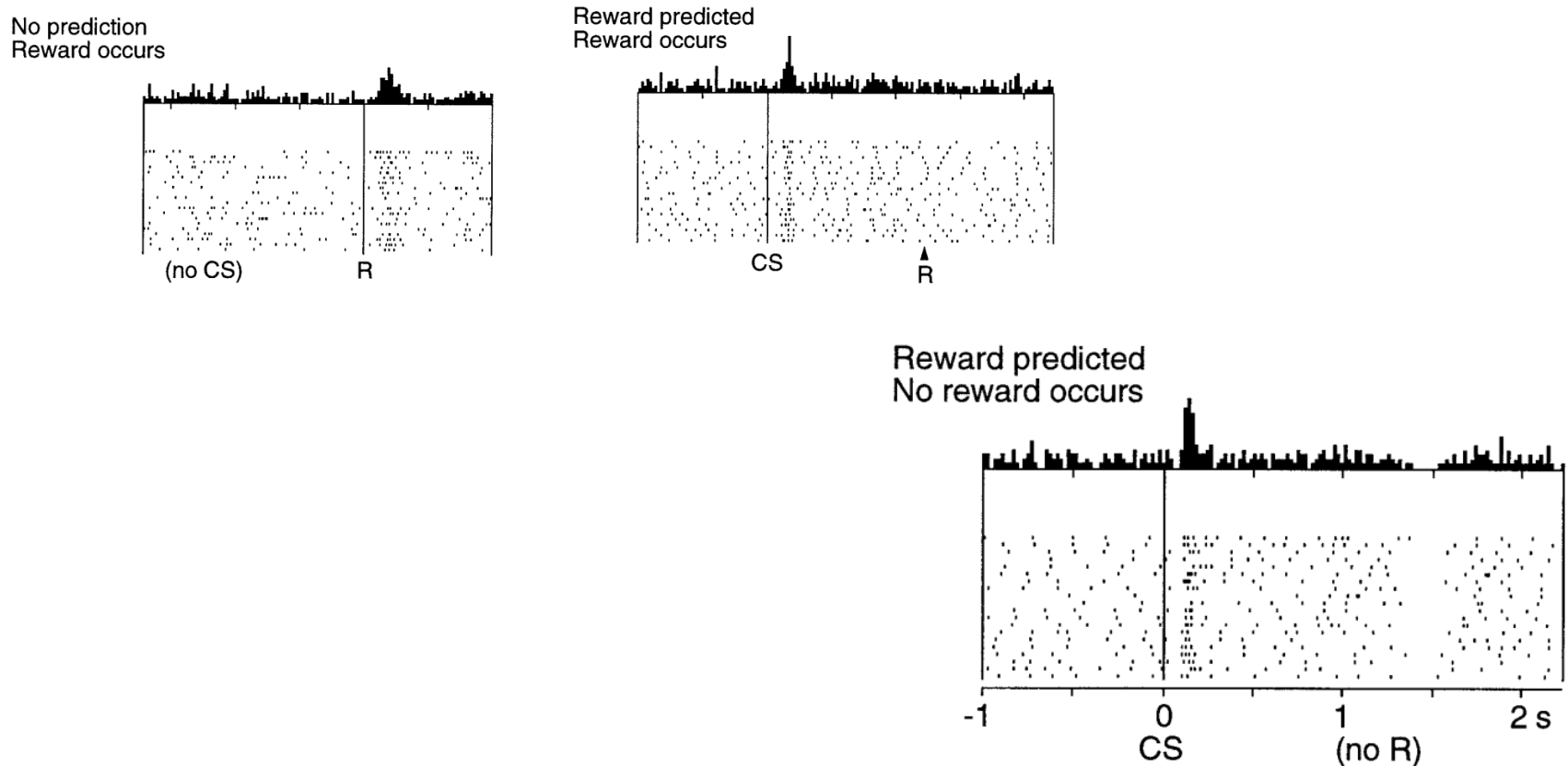


Can shift to earlier prediction

- Shifts back to instruction that predicts trigger that predicts reward



Dopamine response also indicates lack of predicted reward



Dopamine may represent “reward prediction error”

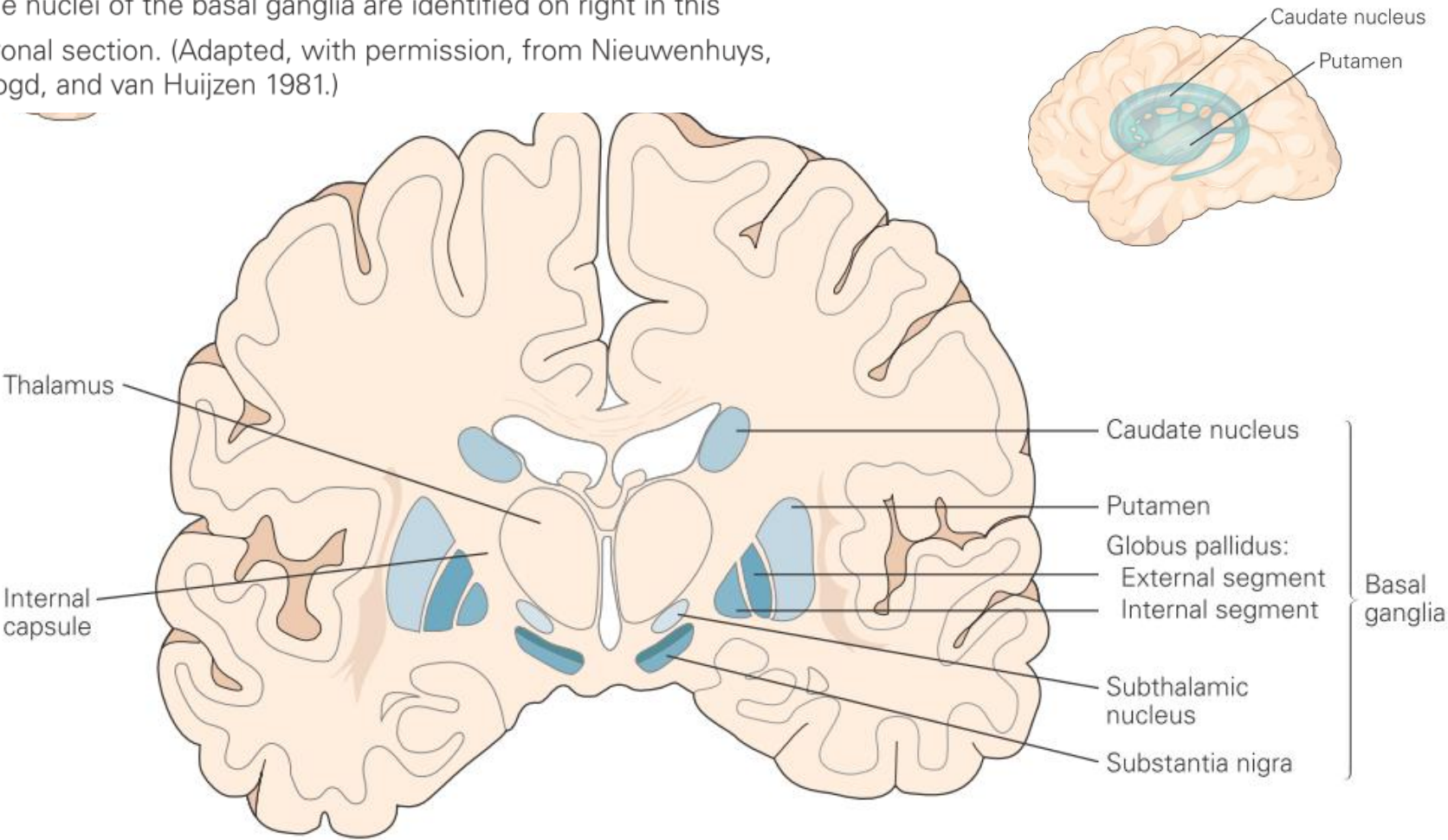
- Difference between expected and predicted reward
- Useful for reinforcement learning
- Learn rewards from this
- Choose actions based on this

Part 6

- Reward prediction in a biological circuit

Figure 43–1 The basal ganglia and surrounding structures.

The nuclei of the basal ganglia are identified on right in this coronal section. (Adapted, with permission, from Nieuwenhuys, Voogd, and van Huijzen 1981.)



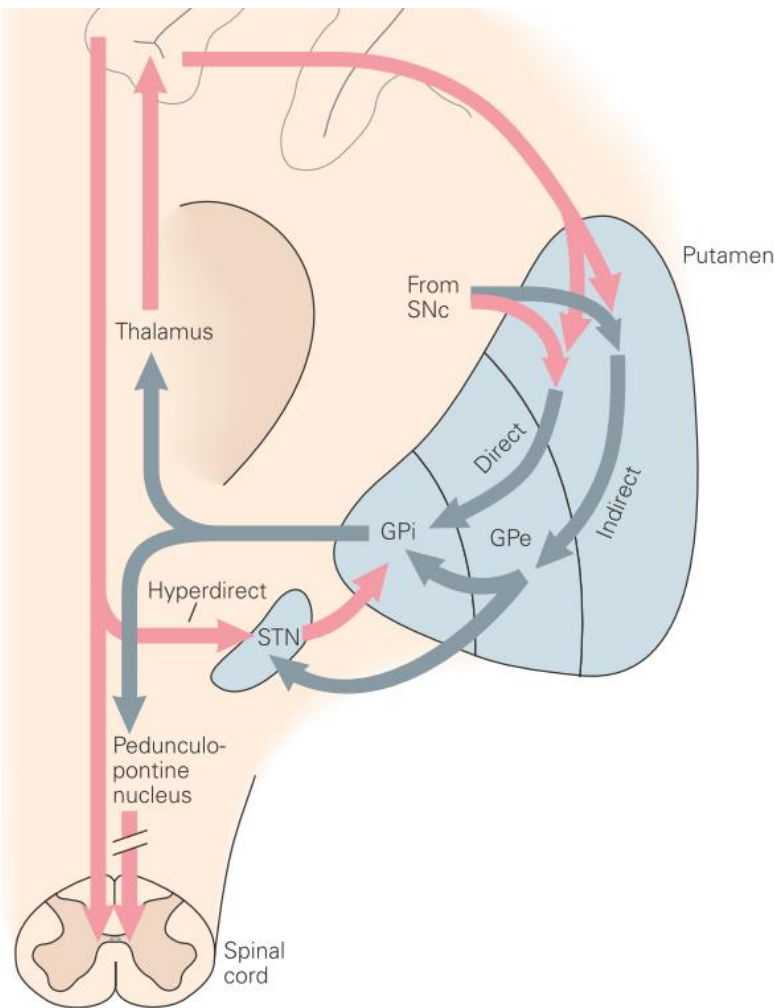


Figure 43–2 The basal ganglia–thalamocortical circuitry. The circuitry of the basal ganglia includes the striatum (here represented by one of its components, the putamen), the external and internal segments of the globus pallidus (GPe and GPi, respectively), the substantia nigra pars reticulata (not shown) and pars compacta (SNc), and the subthalamic nucleus (STN). Cortical input enters the striatum and subthalamic nucleus. Basal ganglia output is conveyed to several thalamic nuclei (the centromedian and parafascicular nuclei and the ventral anterior and ventral lateral nuclei) and the pedunclopontine nucleus. Excitatory connections are shown in **red**, inhibitory pathways in **gray**. The dopaminergic SNc projection to the striatum regulates corticostriatal transmission along direct and indirect pathways.

Dopamine disorders

- Hypo disorders
 - Parkinson's disease
 - Aging
- Hyper disorders
 - Huntington's
 - Tourette's
- Addiction and obsessive behaviors

Parkinson's disease



<https://www.youtube.com/watch?v=pFLC9C-xH8E>

Using movement to help Parkinson's



https://www.youtube.com/watch?v=2sqo5fZ_H5A

Huntington's disease

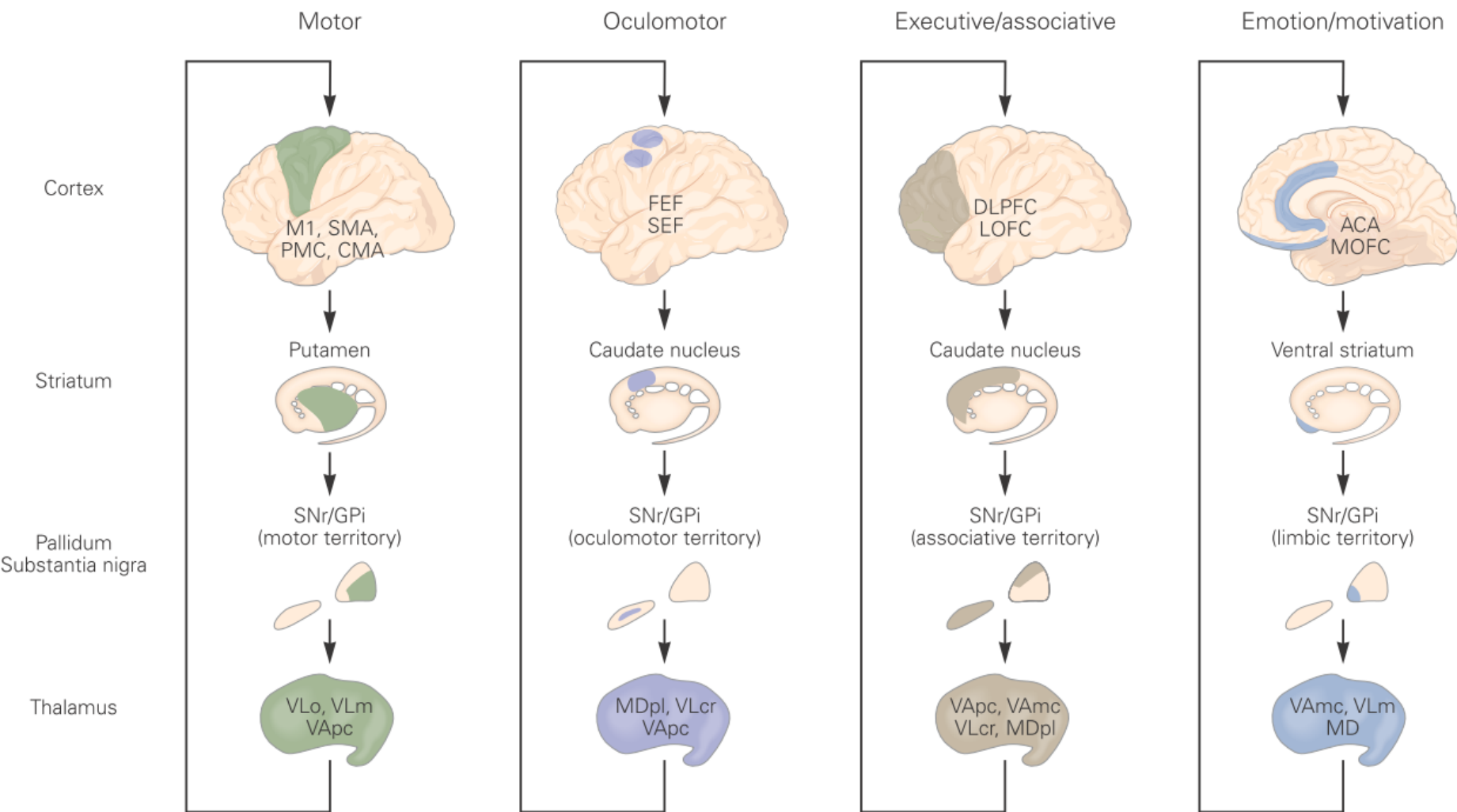


<https://www.youtube.com/watch?v=X5WwnmN2GUU>

A Huntington's disease patient



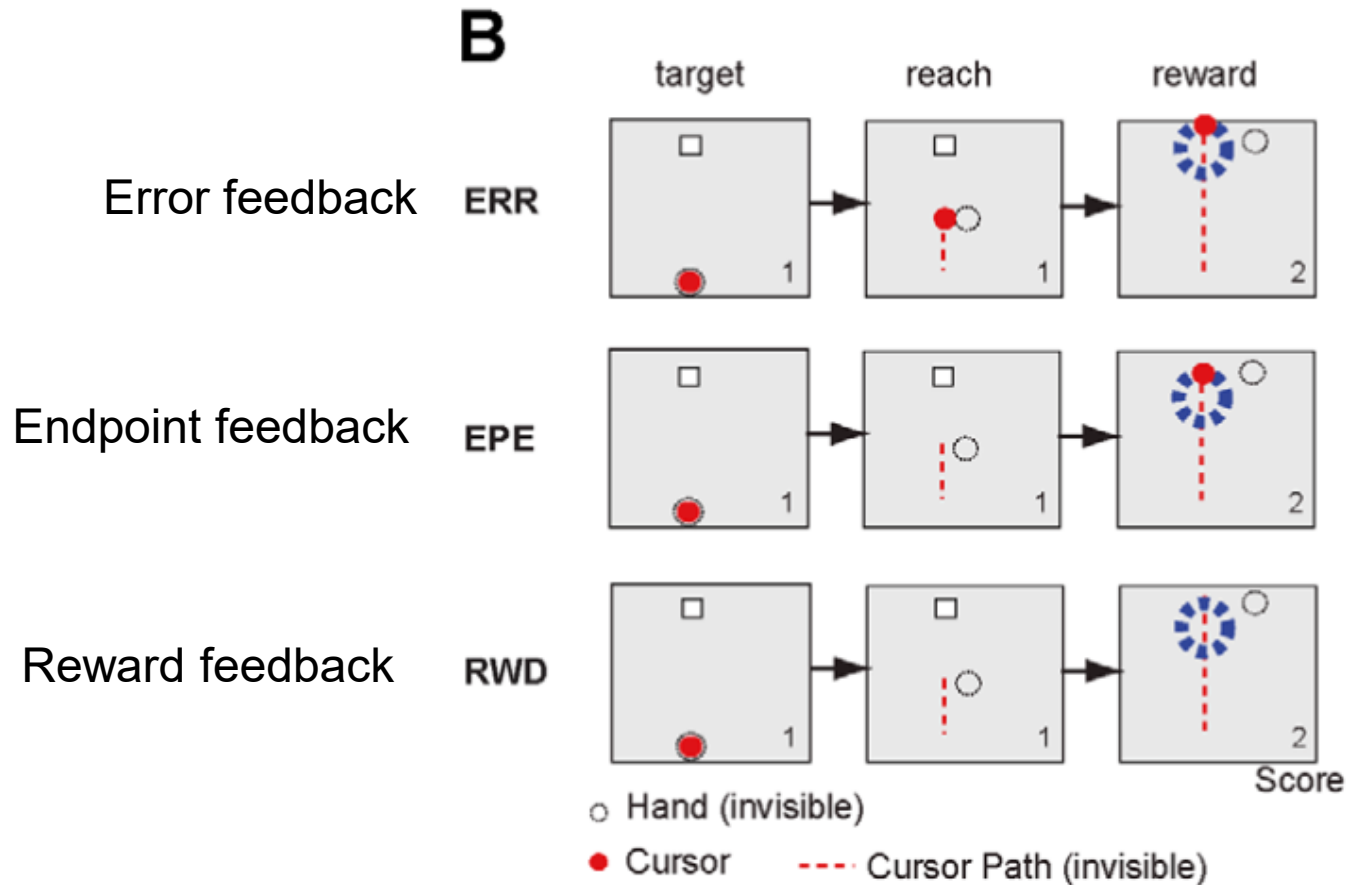
<https://www.youtube.com/watch?v=m6SxJUjJGlc>



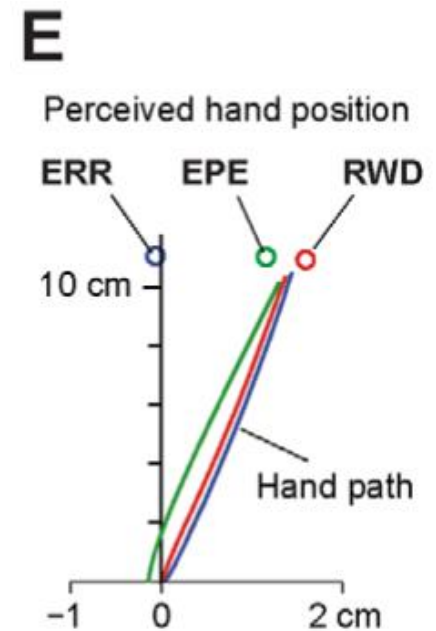
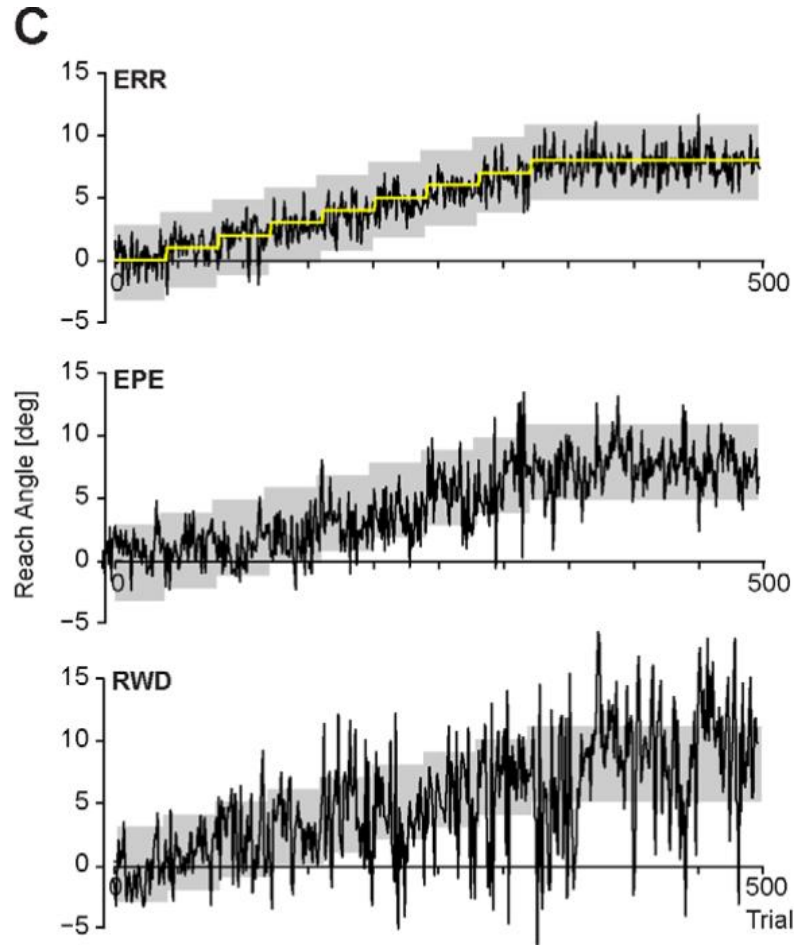
Part 7

- Reward learning in perturbed reaching

Teaching with errors and rewards



Learning from errors and rewards



Part 8

- Modeling reward and error learning

Model the movements

Hand and cursor

$$h^{(k+1)} = u^{(k)} + n_h^{(k)}$$

$$c^{(k)} = h^{(k)} + p^{(k)}$$

$$y^{(k)} = c^{(k)} + n_y^{(k)}$$

h : hand

u : motor command

c : cursor

p : perturbation

y : sensed position

n : noise

Internal estimates

$$\hat{h}^{(k+1)} = \hat{p}^{(k)} + u^{(k)}$$

$$\hat{p}^{(k+1)} = a\hat{p}^{(k)} + n_p^{(k)}$$

a : forgetting factor

Vectorization

$$\mathbf{x}^{(k+1)} = A\mathbf{x}^{(k)} + \mathbf{b}u^{(k)} + \mathbf{n}_x^{(k)}$$

$$y^{(k)} = C\mathbf{x}^{(k)} + n_y^{(k)}$$

$$\mathbf{x}^{(k)} = \begin{pmatrix} p^{(k)} \\ h^{(k)} \end{pmatrix}: \text{state vector}$$

$$\mathbf{n}_x^{(k)} = \begin{pmatrix} n_p^{(k)} \\ n_h^{(k)} \end{pmatrix}$$

$$A = \begin{pmatrix} a & 0 \\ 1 & 0 \end{pmatrix} \quad \mathbf{b} = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \quad C = (0 \quad 1)$$

Kalman filter as error based learning

State equations

$$\mathbf{x}^{(k+1)} = A\mathbf{x}^{(k)} + \mathbf{b}u^{(k)} + \mathbf{n}_x^{(k)}$$

$$y^{(k)} = C\mathbf{x}^{(k)} + n_y^{(k)}$$

$$\mathbf{x}^{(k)} = \begin{pmatrix} p^{(k)} \\ h^{(k)} \end{pmatrix} \quad \mathbf{n}_x^{(k)} = \begin{pmatrix} n_p^{(k)} \\ n_h^{(k)} \end{pmatrix}$$

$$A = \begin{pmatrix} a & 0 \\ 1 & 0 \end{pmatrix} \quad \mathbf{b} = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \quad C = (0 \quad 1)$$

$$\hat{\mathbf{x}}^{(k)} = \begin{pmatrix} \hat{p}^{(k)} \\ \hat{h}^{(k)} \end{pmatrix}$$

Kalman filter

$$\hat{\mathbf{x}}^{(k|k-1)} = A\hat{\mathbf{x}}^{(k-1|k-1)} + \mathbf{b}u^{(k)}$$

$$\hat{\mathbf{x}}^{(k|k)} = \hat{\mathbf{x}}^{(k|k-1)} + K^{(k)} \left(y^{(k)} - C\hat{\mathbf{x}}^{(k|k-1)} \right)$$

$y^{(k)} - C\hat{\mathbf{x}}^{(k|k-1)}$: sensory prediction error

$K^{(k)}$: Kalman gain

Kalman gain

$$K^{(k)} = P^{(k|k-1)} C^T \left(C P^{(k|k-1)} C^T + \sigma_y^2 \right)^{-1}$$

$P^{(k|k-1)}$: Variance of state estimate

Kalman gain controls rate of error based learning

$$\hat{\mathbf{x}}^{(k|k-1)} = A\hat{\mathbf{x}}^{(k-1|k-1)} + \mathbf{b}u^{(k)}$$

$$\hat{\mathbf{x}}^{(k|k)} = \hat{\mathbf{x}}^{(k|k-1)} + K^{(k)} \left(y^{(k)} - C\hat{\mathbf{x}}^{(k|k-1)} \right)$$

Kalman gain

$$K^{(k)} = P^{(k|k-1)} C^T \left(C P^{(k|k-1)} C^T + \sigma_y^2 \right)^{-1}$$

$P^{(k|k-1)}$: Variance of state estimate

- Larger sensory uncertainty
- -> Smaller Kalman gain
- -> Less update of perturbation
- -> Less error based learning
- -> Greater reward based learning

State variance

State equations

$$\mathbf{x}^{(k+1)} = A\mathbf{x}^{(k)} + \mathbf{b}u^{(k)} + \mathbf{n}_x^{(k)}$$

$$y^{(k)} = C\mathbf{x}^{(k)} + n_y^{(k)}$$

Kalman filter

$$\hat{\mathbf{x}}^{(k|k-1)} = A\hat{\mathbf{x}}^{(k-1|k-1)} + \mathbf{b}u^{(k)}$$

$$\hat{\mathbf{x}}^{(k|k)} = \hat{\mathbf{x}}^{(k|k-1)} + K^{(k)} \left(y^{(k)} - C\hat{\mathbf{x}}^{(k|k-1)} \right)$$

$$K^{(k)} = P^{(k|k-1)} C^T \left(C P^{(k|k-1)} C^T + \sigma_y^2 \right)^{-1}$$

Variance propagation

$$P^{(k|k-1)} = A P^{(k-1|k-1)} A^T + Q^{(k)}$$

$$P^{(k|k)} = \left(I - K^{(k)} C \right) P^{(k|k-1)}$$

$$Q^{(k)} = \begin{pmatrix} \sigma_h^2 & 0 \\ 0 & \sigma_p^2 \end{pmatrix} : \text{variance of state variables}$$

Credit assignment problem

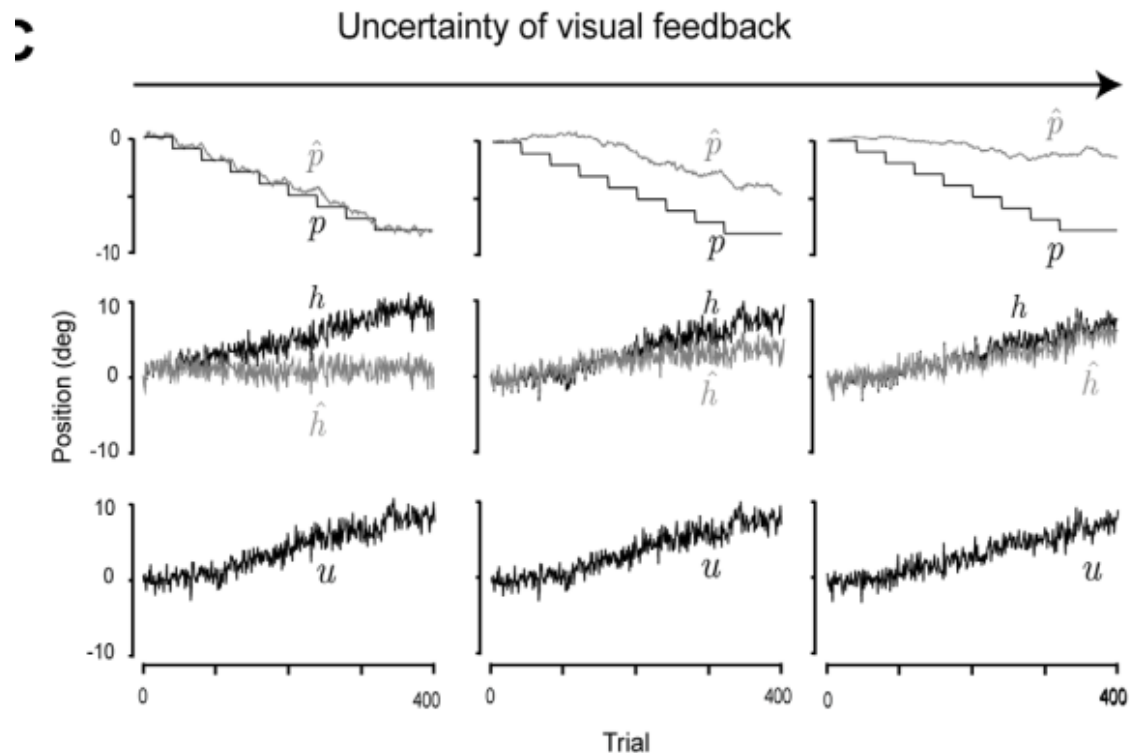
$$\mathbf{x}^{(k+1)} = A\mathbf{x}^{(k)} + \mathbf{b}u^{(k)} + \mathbf{n}_x^{(k)}$$

$$y^{(k)} = C\mathbf{x}^{(k)} + n_y^{(k)}$$

$$\hat{\mathbf{x}}^{(k|k-1)} = A\hat{\mathbf{x}}^{(k-1|k-1)} + \mathbf{b}u^{(k)}$$

$$\hat{\mathbf{x}}^{(k|k)} = \hat{\mathbf{x}}^{(k|k-1)} + K^{(k)} \left(y^{(k)} - C\hat{\mathbf{x}}^{(k|k-1)} \right)$$

$$K^{(k)} = P^{(k|k-1)} C^T \left(C P^{(k|k-1)} C^T + \sigma_y^2 \right)^{-1}$$



Deciding on the movement

Control policy

$$u^{(k)} = -\hat{p}^{(k)} + w_r^{(k)} + n_u^{(k)}$$

$w_r^{(k)}$: reward-maximizing policy

State equations

$$\mathbf{x}^{(k+1)} = A\mathbf{x}^{(k)} + \mathbf{b}u^{(k)} + \mathbf{n}_x^{(k)}$$

$$y^{(k)} = C\mathbf{x}^{(k)} + n_y^{(k)}$$

Kalman filter

$$\hat{\mathbf{x}}^{(k|k-1)} = A\hat{\mathbf{x}}^{(k-1|k-1)} + \mathbf{b}u^{(k)}$$

$$\hat{\mathbf{x}}^{(k|k)} = \hat{\mathbf{x}}^{(k|k-1)} + K^{(k)} \left(y^{(k)} - C\hat{\mathbf{x}}^{(k|k-1)} \right)$$

$$K^{(k)} = P^{(k|k-1)} C^T \left(C P^{(k|k-1)} C^T + \sigma_y^2 \right)^{-1}$$

Variance propagation

$$P^{(k|k-1)} = A P^{(k-1|k-1)} A^T + Q^{(k)}$$

$$P^{(k|k)} = \left(I - K^{(k)} C \right) P^{(k|k-1)}$$

Reward based learning

State equations

$$\mathbf{x}^{(k+1)} = A\mathbf{x}^{(k)} + \mathbf{b}u^{(k)} + \mathbf{n}_x^{(k)}$$

$$y^{(k)} = C\mathbf{x}^{(k)} + n_y^{(k)}$$

Kalman filter

$$\hat{\mathbf{x}}^{(k|k-1)} = A\hat{\mathbf{x}}^{(k-1|k-1)} + \mathbf{b}u^{(k)}$$

$$\hat{\mathbf{x}}^{(k|k)} = \hat{\mathbf{x}}^{(k|k-1)} + K^{(k)} \left(y^{(k)} - C\hat{\mathbf{x}}^{(k|k-1)} \right)$$

Control policy

$$u^{(k)} = -\hat{p}^{(k)} + w_r^{(k)} + n_u^{(k)}$$

Policy learning

$$w_r^{(k+1)} = w_r^{(k)} + \alpha_r \delta^{(k)} n_u^{(k)}$$

$$\delta^{(k)} = r^{(k)} + \gamma \hat{V}^{(k+1)} - \hat{V}^{(k)}$$

$\delta^{(k)}$: reward prediction error

$V^{(k)}$: Discounted accumulated reward

$\hat{V}^{(k)}$: Estimate of accumulated reward

γ : Discounting factor

Reward learning

$$V^{(k)} = E \left[r^{(k)} + \gamma r^{(k+1)} + \gamma^2 r^{(k+2)} + \dots \right]$$

$$\hat{V}^{(k+1)} = \hat{V}^{(k)} + \alpha_V \delta^{(k)}$$

Internal model learning

Policy learning

$$w_r^{(k+1)} = w_r^{(k)} + \alpha_r \delta^{(k)} n_u^{(k)}$$

$$\delta^{(k)} = r^{(k)} + \gamma \hat{V}^{(k+1)} - \hat{V}^{(k)}$$

$\delta^{(k)}$: reward prediction error

$V^{(k)}$: Discounted accumulated reward

$\hat{V}^{(k)}$: Estimate of accumulated reward

γ : Discounting factor

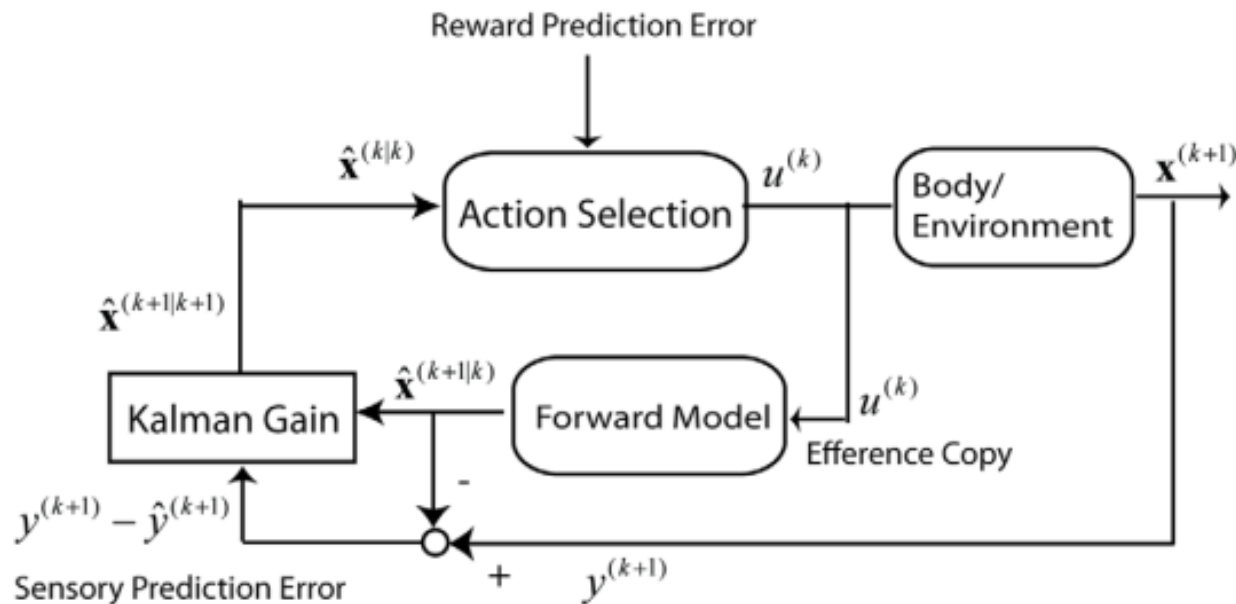
- Choose movement
- Observe reward
- Compute δ_k
 - Using old $\hat{V}^{(k+1)}$
- Update w_r and $\hat{V}^{(k+1)}$

Reward learning

$$V^{(k)} = E \left[r^{(k)} + \gamma r^{(k+1)} + \gamma^2 r^{(k+2)} + \dots \right]$$

$$\hat{V}^{(k+1)} = \hat{V}^{(k)} + \alpha_V \delta^{(k)}$$

Box and arrow diagram of the model



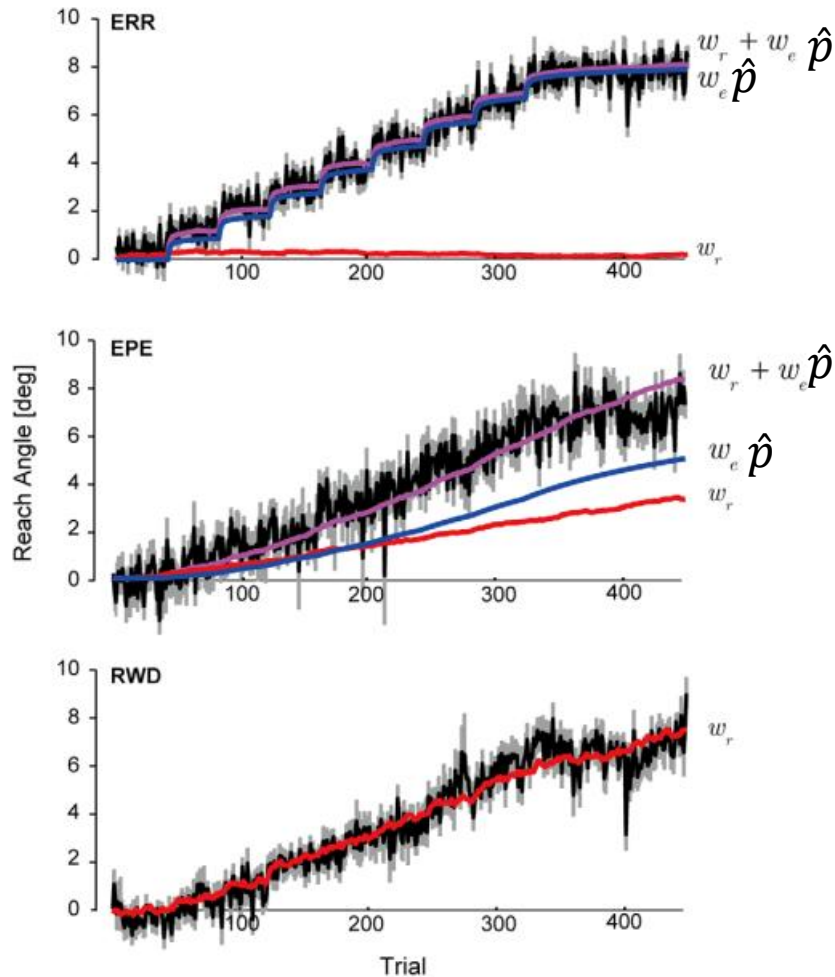
- Main loop
 - Choose movement
 - Generate actual and estimated state
 - Calculate error and reward
 - Update controller

$$u^{(k)} = w_e^{(k)} + w_r^{(k)} + n_u^{(k)}$$

$$\text{Error-based learning: } w_e^{(k)} = -\hat{p}^{(k)}$$

$$\text{Reward-based learning: } \Delta w_r^{(k)} = \alpha_r \delta^{(k)} n_u^{(k)}$$

Model successfully learns from error or reward



In class

- Use the equations in the lecture to reproduce the figure on the previous slide.